Slip 1

Q.1) Declare a class Student containing roll_no and percentage. Accept data for 2 objects and display the roll_no with the highest percentage.

```
#include <iostream>
using namespace std;

class Student {
public:
    int roll_no;
    float percentage;
    void accept() {
        cout << "Enter Roll No and Percentage: ";
        cin >> roll_no >> percentage;
    }
};

int main() {
    Student s1, s2;
    s1.accept();
    s2.accept();
    if (s1.percentage > s2.percentage)
        cout << "Highest Percentage Roll No: " << s1.roll_no << endl;
    else
        cout << "Highest Percentage Roll No: " << s2.roll_no << endl;
    return 0;
}
```

Q.2) Accept an array of integers, compute the square root of each element, and throw/catch an exception if a negative number is encountered.

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    for (int i = 0; i < n; i++) {
        try {
            if (arr[i] < 0) throw arr[i];
            cout << "Sqrt of " << arr[i] << " = " << sqrt(arr[i]) << endl;
        } catch (int neg) {
            cout << "Exception: Negative number (" << neg << ") encountered!\n";
        }
    }
    return 0;
```

```
}
```

Slip 2

Q.1) Create an inline function that returns the length of a given string.

```
#include <iostream>
#include <string>
using namespace std;

inline int getLength(string str) { return str.length(); }

int main() {
    string s;
    cout << "Enter string: ";
    getline(cin, s);
    cout << "Length: " << getLength(s) << endl;
    return 0;
}
```

Q.2) Define a class Bus and write a menu-driven program using manipulators to accept n buses, display all buses, and display buses from a specified starting and ending destination.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Bus {
public:
    int bno, seats;
    string bname, start, dest;
    void accept() {
        cout << "Enter Bus No, Name, Seats, Start, Destination: ";
        cin >> bno >> bname >> seats >> start >> dest;
    }
    void display() {
        cout << left << setw(10) << bno << setw(15) << bname
             << setw(10) << seats << setw(15) << start << setw(15) << dest << endl;
    }
};

int main() {
    int n = 0, ch;
    Bus b[50];
    do {
        cout << "\n1.Accept  2.Display All  3.Search Route  4.Exit\nChoice: ";
        cin >> ch;
        if (ch == 1) {
            cout << "How many buses? "; cin >> n;
            for (int i = 0; i < n; i++) b[i].accept();
        } else if (ch == 2) {
```

```
            for (int i = 0; i < n; i++) b[i].display();
        } else if (ch == 3) {
            string s, d;
            cout << "Enter Start and Destination: "; cin >> s >> d;
            for (int i = 0; i < n; i++)
                if (b[i].start == s && b[i].dest == d) b[i].display();
        }
    } while (ch != 4);
    return 0;
}
```

Slip 3

Q.1) Read Book.txt file and display Books data on the screen.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream fin("Book.txt");
    if (!fin) { cout << "File not found!\n"; return 1; }
    string line;
    while (getline(fin, line)) cout << line << endl;
    fin.close();
    return 0;
}
```

Q.2) Define class Point (x, y). Use default, parameterized, and copy constructors to copy one object to another.

```
#include <iostream>
using namespace std;

class Point {
    int x, y;
public:
    Point() : x(0), y(0) {}
    Point(int a, int b) : x(a), y(b) {}
    Point(const Point &p) { x = p.x; y = p.y; }
    void display() { cout << "Point(" << x << ", " << y << ")\n"; }
};

int main() {
    Point p1;
    Point p2(10, 20);
    Point p3(p2); // Copy constructor
    p1.display();
    p2.display();
    p3.display();
    return 0;
}
```

Slip 4

Q.1) Use unordered_map to store and display the names of 3 students with their roll numbers.

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

int main() {
    unordered_map<int, string> students;
    for (int i = 0; i < 3; i++) {
        int r; string name;
        cout << "Enter Roll No and Name: ";
        cin >> r >> name;
        students[r] = name;
    }
    cout << "\nStudent List:\n";
    for (auto &p : students)
        cout << "Roll No: " << p.first << ", Name: " << p.second << endl;
    return 0;
}
```

Q.2) Create class Person with passport details. Use function overloading and an array of objects to enter details, display one person, and display all persons.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
public:
    int pid;
    string name, nat, gen, dob, doi, doe;
    void accept() {
        cout << "Enter Passport ID, Name, Nationality, Gender, DOB, IssueDate,
ExpiryDate: ";
        cin >> pid >> name >> nat >> gen >> dob >> doi >> doe;
    }
    void show() {
        cout << pid << " | " << name << " | " << nat << " | " << gen
             << " | " << dob << " | " << doi << " | " << doe << endl;
    }
};

void display(Person p[], int n) {
    for (int i = 0; i < n; i++) p[i].show();
}

void display(Person p[], int n, int search_id) {
    for (int i = 0; i < n; i++)
```

```
        if (p[i].pid == search_id) p[i].show();
}

int main() {
    int n, id;
    cout << "Enter number of persons: "; cin >> n;
    Person p[n];
    for (int i = 0; i < n; i++) p[i].accept();
    cout << "\n--- All Passports ---\n";
    display(p, n);
    cout << "\nEnter Passport ID to search: "; cin >> id;
    display(p, n, id);
    return 0;
}
```

Slip 5

Q.1) Define class Cube (length, width, depth) and function volume() to calculate volume.

```
#include <iostream>
using namespace std;

class Cube {
    double l, w, d;
public:
    void accept() {
        cout << "Enter length, width, depth: ";
        cin >> l >> w >> d;
    }
    double volume() { return l * w * d; }
};

int main() {
    Cube c;
    c.accept();
    cout << "Volume: " << c.volume() << endl;
    return 0;
}
```

Q.2) Simple Calculator (+, -, *, /) that throws an exception for division by zero or invalid operator.

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    char op;
    cout << "Enter expression (e.g. 10 / 2): ";
    cin >> a >> op >> b;
    try {
        if (op != '+' && op != '-' && op != '*' && op != '/')
```

```
            throw string("Invalid Operator!");
        if (op == '/' && b == 0)
            throw 0;
        switch (op) {
            case '+': cout << "Result: " << a + b << endl; break;
            case '-': cout << "Result: " << a - b << endl; break;
            case '*': cout << "Result: " << a * b << endl; break;
            case '/': cout << "Result: " << a / b << endl; break;
        }
    } catch (string &msg) {
        cout << "Error: " << msg << endl;
    } catch (int) {
        cout << "Error: Division by zero!\n";
    }
    return 0;
}
```

Slip 6

Q.1) Calculate simple interest using a class with a parameterized constructor having a default value for rate.

```
#include <iostream>
using namespace std;

class Interest {
    float p, n, r;
public:
    Interest(float pr, float yr, float rate = 8.5) : p(pr), n(yr), r(rate) {}
    void calc() { cout << "Simple Interest: " << (p * n * r) / 100.0 << endl; }
};

int main() {
    Interest i1(10000, 2);
    i1.calc();
    return 0;
}
```

Q.2) Create class Date (dd, mm, yyyy) and overload >> and << operators to accept and display a date.

```
#include <iostream>
using namespace std;

class Date {
    int dd, mm, yyyy;
public:
    friend istream& operator>>(istream &in, Date &d) {
        cout << "Enter DD MM YYYY: ";
        in >> d.dd >> d.mm >> d.yyyy;
        return in;
    }
    friend ostream& operator<<(ostream &out, const Date &d) {
```

```
        out << d.dd << "/" << d.mm << "/" << d.yyyy;
        return out;
    }
};

int main() {
    Date d;
    cin >> d;
    cout << "Date is: " << d << endl;
    return 0;
}
```

Slip 7

Q.1) Base class Product (Product_Id, Product_Name, Price), derived class Discount (Discount_In_Percentage). Calculate total price and discount for n products using manipulators.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Product {
public:
    int pid; string pname; float price;
};

class Discount : public Product {
public:
    float disc_pct, disc_amt;
    void accept() {
        cout << "Enter ID, Name, Price, Discount(%): ";
        cin >> pid >> pname >> price >> disc_pct;
        disc_amt = (price * disc_pct) / 100.0;
    }
    void display() {
        cout << left << setw(8) << pid << setw(15) << pname
             << setw(10) << price << setw(10) << disc_amt << endl;
    }
};

int main() {
    int n; float tot_p = 0, tot_d = 0;
    cout << "Enter number of products: "; cin >> n;
    Discount d[n];
    for (int i = 0; i < n; i++) {
        d[i].accept();
        tot_p += d[i].price;
        tot_d += d[i].disc_amt;
    }
    cout << "\nID      Name           Price     Discount\n";
```

```
    for (int i = 0; i < n; i++) d[i].display();
    cout << "Total Price: " << tot_p << "\nTotal Discount: " << tot_d
         << "\nFinal Bill: " << tot_p - tot_d << endl;
    return 0;
}
```

Q.2) Append the contents of one file to another file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream fin("source.txt");
    ofstream fout("dest.txt", ios::app);
    string line;
    while (getline(fin, line)) fout << line << "\n";
    cout << "Content appended successfully.\n";
    fin.close(); fout.close();
    return 0;
}
```

Slip 8

Q.1) Define class Rectangle (length, breadth). Accept data for one object and display area and perimeter.

```
#include <iostream>
using namespace std;

class Rectangle {
    float l, b;
public:
    void accept() { cout << "Enter length and breadth: "; cin >> l >> b; }
    void display() {
        cout << "Area: " << l * b << "\nPerimeter: " << 2 * (l + b) << endl;
    }
};

int main() {
    Rectangle r;
    r.accept();
    r.display();
    return 0;
}
```

Q.2) Combine two strings and show the execution of a dynamic constructor using class Mystring (name, length)

```
#include <iostream>
#include <cstring>
using namespace std;
```

```
class Mystring {
    char *name;
    int length;
public:
    Mystring() { length = 0; name = new char[1]; name[0] = '\0'; }
    Mystring(const char *s) {
        length = strlen(s);
        name = new char[length + 1];
        strcpy(name, s);
    }
    void join(Mystring &a, Mystring &b) {
        length = a.length + b.length;
        delete[] name;
        name = new char[length + 1];
        strcpy(name, a.name);
        strcat(name, b.name);
    }
    void display() { cout << name << " (Length: " << length << ")\n"; }
};

int main() {
    Mystring s1("Hello "), s2("World"), s3;
    s3.join(s1, s2);
    s3.display();
    return 0;
}
```

Slip 9

Q.1) Declare class Product (product_code, name, price). Accept and display information for 2 objects.

```
#include <iostream>
#include <string>
using namespace std;

class Product {
    int code; string name; float price;
public:
    void accept() { cout << "Enter Code, Name, Price: "; cin >> code >> name >>
price; }
    void display() { cout << code << " - " << name << " : Rs." << price << endl; }
};

int main() {
    Product p1, p2;
    p1.accept(); p2.accept();
    p1.display(); p2.display();
    return 0;
}
```

Q.2) Write function vowelwords() that reads FIRST.TXT and creates SECOND.TXT

containing only words starting with a lowercase vowel (a, e, i, o, u).

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void vowelwords() {
    ifstream fin("FIRST.TXT");
    ofstream fout("SECOND.TXT");
    string w;
    while (fin >> w) {
        char c = w[0];
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
            fout << w << " ";
    }
    fin.close(); fout.close();
    cout << "Vowel words written to SECOND.TXT\n";
}

int main() {
    vowelwords();
    return 0;
}
```

Slip 10

Q.1) Class Mobile (Mobile_Id, Mobile_Name, Mobile_Price). Initialize using parameterized constructor and display price right-justified with 2-digit precision.

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

class Mobile {
    int id; string name; double price;
public:
    Mobile(int i, string n, double p) : id(i), name(n), price(p) {}
    void display() {
        cout << "ID: " << id << " | Name: " << name << " | Price: "
             << right << setw(10) << fixed << setprecision(2) << price << endl;
    }
};

int main() {
    Mobile m(101, "Samsung", 24999.5);
    m.display();
    return 0;
}
```

Q.2) Base class Shape, derived classes Circle, Rectangle, and Triangle. Calculate area using a pure virtual function.

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() = 0;
};

class Circle : public Shape {
    float r;
public:
    Circle(float rad) : r(rad) {}
    void area() override { cout << "Circle Area: " << 3.14159 * r * r << endl; }
};

class Rectangle : public Shape {
    float l, b;
public:
    Rectangle(float x, float y) : l(x), b(y) {}
    void area() override { cout << "Rectangle Area: " << l * b << endl; }
};

class Triangle : public Shape {
    float b, h;
public:
    Triangle(float x, float y) : b(x), h(y) {}
    void area() override { cout << "Triangle Area: " << 0.5 * b * h << endl; }
};

int main() {
    Circle c(5); Rectangle r(4, 6); Triangle t(3, 8);
    c.area(); r.area(); t.area();
    return 0;
}
```

Slip 11

Q.1) Class Person (name, age), derived class Student (roll_no, marks). Display all info using a function.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
protected:
    string name; int age;
};

class Student : public Person {
    int roll_no; float marks;
public:
```

```
    void accept() {
        cout << "Enter Name, Age, Roll No, Marks: ";
        cin >> name >> age >> roll_no >> marks;
    }
    void display() {
        cout << "Name: " << name << ", Age: " << age
             << ", Roll: " << roll_no << ", Marks: " << marks << endl;
    }
};

int main() {
    Student s;
    s.accept();
    s.display();
    return 0;
}
```

Q.2) Read Item information (Itemno, Itemname, Itemprice, Quantity) of n items and write it to a file.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    int n, no, qty; string name; float price;
    ofstream fout("items.txt");
    cout << "Enter number of items: "; cin >> n;
    for (int i = 0; i < n; i++) {
        cout << "Enter Item No, Name, Price, Quantity: ";
        cin >> no >> name >> price >> qty;
        fout << no << " " << name << " " << price << " " << qty << "\n";
    }
    fout.close();
    cout << "Items saved to items.txt\n";
    return 0;
}
```

Slip 12

Q.1) Print area of a circle and square using inline functions.

```
#include <iostream>
using namespace std;

inline float areaCircle(float r) { return 3.14159f * r * r; }
inline float areaSquare(float s) { return s * s; }

int main() {
    cout << "Circle Area (r=5): " << areaCircle(5) << endl;
    cout << "Square Area (s=4): " << areaSquare(4) << endl;
    return 0;
```

```
}
```

Q.2) Class Date (dd, mm, yyyy). Initialize using parameterized constructor, validate month, and display in dd-Mon-yyyy format.

```
#include <iostream>
using namespace std;

class Date {
    int d, m, y;
public:
    Date(int dd, int mm, int yyyy) : d(dd), m(mm), y(yyyy) {}
    void display() {
        const char* months[] = {"", "Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};
        if (m < 1 || m > 12) cout << "Invalid Month!\n";
        else cout << d << "-" << months[m] << "-" << y << endl;
    }
};

int main() {
    int d, m, y;
    cout << "Enter DD MM YYYY: "; cin >> d >> m >> y;
    Date dt(d, m, y);
    dt.display();
    return 0;
}
```

Slip 13

Q.1) Calculate area of cone, sphere, and circle using function overloading.

```
#include <iostream>
using namespace std;

float area(float r, float l) { return 3.14159f * r * (r + l); } // Cone
double area(double r) { return 4 * 3.14159 * r * r; }          // Sphere
float area(float r) { return 3.14159f * r * r; }               // Circle

int main() {
    cout << "Cone Area: " << area(3.0f, 5.0f) << endl;
    cout << "Sphere Area: " << area(4.0) << endl;
    cout << "Circle Area: " << area(4.0f) << endl;
    return 0;
}
```

Q.2) Student Grading System: Accept name and marks; throw exceptions for marks > 100 or < 0, or empty name.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
```

```
    string name; int marks;
    cout << "Enter Student Name: "; getline(cin, name);
    cout << "Enter Marks: "; cin >> marks;
    try {
        if (name.empty()) throw string("Name cannot be empty!");
        if (marks < 0 || marks > 100) throw marks;
        cout << "Student: " << name << " | Marks: " << marks << endl;
    } catch (string &e) {
        cout << "Exception: " << e << endl;
    } catch (int m) {
        cout << "Exception: Invalid Marks (" << m << ")!\n";
    }
    return 0;
}
```

Slip 14

Q.1) Find the sum of numbers between 1 to n using a constructor where n is passed to the constructor.

```
#include <iostream>
using namespace std;

class Sum {
public:
    Sum(int n) {
        int s = (n * (n + 1)) / 2;
        cout << "Sum from 1 to " << n << " is: " << s << endl;
    }
};

int main() {
    int n;
    cout << "Enter n: "; cin >> n;
    Sum obj(n);
    return 0;
}
```

Q.2) Class College (College_Id, College_Name, Establishment_year, University_Name). Use an array of objects and function overloading to display by University or by Establishment year.

```
#include <iostream>
#include <string>
using namespace std;

class College {
public:
    int cid, year;
    string cname, uname;
    void accept() {
        cout << "Enter ID, Name, Est_Year, University: ";
        cin >> cid >> cname >> year >> uname;
```

```
    }
    void show() { cout << cid << " | " << cname << " | " << year << " | " << uname
<< endl; }
};

void display(College c[], int n, string uni) {
    for (int i = 0; i < n; i++) if (c[i].uname == uni) c[i].show();
}

void display(College c[], int n, int yr) {
    for (int i = 0; i < n; i++) if (c[i].year == yr) c[i].show();
}

int main() {
    int n, yr; string uni;
    cout << "Enter number of colleges: "; cin >> n;
    College c[n];
    for (int i = 0; i < n; i++) c[i].accept();
    cout << "Enter University to search: "; cin >> uni;
    display(c, n, uni);
    cout << "Enter Year to search: "; cin >> yr;
    display(c, n, yr);
    return 0;
}
```

Slip 15

Q.1) Create an inline function to calculate the area of a rectangle with a default value for width.

```
#include <iostream>
using namespace std;

inline float rectArea(float l, float w = 5.0f) { return l * w; }

int main() {
    cout << "Area (default width 5): " << rectArea(10) << endl;
    cout << "Area (width 4): " << rectArea(10, 4) << endl;
    return 0;
}
```

Q.2) Base classes Employee (Name, Designation) and Project (Project_Id, title). Derive Emp_Proj (Duration). Menu-driven program to build/display master table and sort by duration.

```
#include <iostream>
#include <algorithm>
#include <string>
using namespace std;

class Employee { public: string name, desig; };
class Project { public: int pid; string title; };
```

```
class Emp_Proj : public Employee, public Project {
public:
    int duration;
    void accept() {
        cout << "Enter Emp Name, Designation, Proj ID, Title, Duration: ";
        cin >> name >> desig >> pid >> title >> duration;
    }
    void display() {
        cout << name << " | " << desig << " | " << pid << " | " << title << " | "
<< duration << "m\n";
    }
};

int main() {
    int n = 0, ch;
    Emp_Proj ep[50];
    do {
        cout << "\n1.Build  2.Display  3.Sort by Duration  4.Exit\nChoice: ";
        cin >> ch;
        if (ch == 1) {
            cout << "How many records? "; cin >> n;
            for (int i = 0; i < n; i++) ep[i].accept();
        } else if (ch == 2) {
            for (int i = 0; i < n; i++) ep[i].display();
        } else if (ch == 3) {
            sort(ep, ep + n, [](Emp_Proj a, Emp_Proj b) { return a.duration <
b.duration; });
            for (int i = 0; i < n; i++) ep[i].display();
        }
    } while (ch != 4);
    return 0;
}
```

Slip 16

Q.1) Count the frequency of each character in a user-entered string using unordered_map.

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

int main() {
    string s;
    cout << "Enter string: "; getline(cin, s);
    unordered_map<char, int> freq;
    for (char c : s) freq[c]++;
    for (auto &p : freq) cout << "'" << p.first << "' : " << p.second << endl;
    return 0;
}
```

Q.2) Class Employee (E_no, E_Name, Designation, Salary). Accept 2 employees and display the one with maximum salary using this pointer.

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
    int eno; string name, desig; float sal;
public:
    void accept() {
        cout << "Enter E_No, Name, Designation, Salary: ";
        cin >> eno >> name >> desig >> sal;
    }
    Employee compare(Employee e) {
        return (this->sal > e.sal) ? *this : e;
    }
    void display() {
        cout << "Max Salary Employee: " << eno << " | " << name << " | Rs." << sal
<< endl;
    }
};

int main() {
    Employee e1, e2;
    e1.accept(); e2.accept();
    Employee maxEmp = e1.compare(e2);
    maxEmp.display();
    return 0;
}
```

Slip 17

Q.1) Count the number of words in a given file.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream fin("sample.txt");
    string word; int count = 0;
    while (fin >> word) count++;
    cout << "Total words: " << count << endl;
    fin.close();
    return 0;
}
```

Q.2) Multilevel inheritance: Flight $\rightarrow$ Route $\rightarrow$ Reservation. Enter n reservations, display all, and display Business class reservations.

```
#include <iostream>
#include <string>
```

```
using namespace std;

class Flight { protected: int fno; string fname; };
class Route : public Flight { protected: string src, dest; };

class Reservation : public Route {
public:
    int seats; string rclass, date; float fare;
    void accept() {
        cout << "Enter FlightNo, Name, Src, Dest, Seats, Class, Fare, Date: ";
        cin >> fno >> fname >> src >> dest >> seats >> rclass >> fare >> date;
    }
    void display() {
        cout << fno << " | " << fname << " | " << src << "->" << dest
             << " | Seats:" << seats << " | " << rclass << " | Rs." << fare << " |
" << date << endl;
    }
};

int main() {
    int n; cout << "Enter number of reservations: "; cin >> n;
    Reservation r[n];
    for (int i = 0; i < n; i++) r[i].accept();
    cout << "\n--- All Reservations ---\n";
    for (int i = 0; i < n; i++) r[i].display();
    cout << "\n--- Business Class Reservations ---\n";
    for (int i = 0; i < n; i++)
        if (r[i].rclass == "Business") r[i].display();
    return 0;
}
```

Slip 18

Q.1) Class Product (productCode, productName, weight). If weight > 100, throw an exception; otherwise display details.

```
#include <iostream>
#include <string>
using namespace std;

class Product {
    int code; string name; float weight;
public:
    void accept() {
        cout << "Enter Code, Name, Weight: ";
        cin >> code >> name >> weight;
        if (weight > 100) throw weight;
        cout << "Product: " << code << " | " << name << " | " << weight << "kg\n";
    }
};

int main() {
```

```
    Product p;
    try {
        p.accept();
    } catch (float w) {
        cout << "Error: Weight (" << w << ") exceeds 100 limit!\n";
    }
    return 0;
}
```

Q.2) Class Distance (feet, inches). Write a friend function to add two Distance objects.

```
#include <iostream>
using namespace std;

class Distance {
    int feet, inches;
public:
    Distance(int f = 0, int i = 0) : feet(f), inches(i) {}
    friend Distance add(Distance d1, Distance d2) {
        Distance temp;
        temp.inches = d1.inches + d2.inches;
        temp.feet = d1.feet + d2.feet + (temp.inches / 12);
        temp.inches %= 12;
        return temp;
    }
    void display() { cout << feet << " ft " << inches << " in\n"; }
};

int main() {
    Distance d1(5, 8), d2(3, 9), d3;
    d3 = add(d1, d2);
    d3.display();
    return 0;
}
```

Slip 19

Q.1) Class Integer. Overload unary pre and post decrement -- operators.

```
#include <iostream>
using namespace std;

class Integer {
    int x;
public:
    Integer(int val) : x(val) {}
    void operator--() { --x; }       // Pre-decrement
    void operator--(int) { x--; }    // Post-decrement
    void display() { cout << "Value: " << x << endl; }
};

int main() {
```

```
    Integer num(10);
    --num; num.display();
    num--; num.display();
    return 0;
}
```

Q.2) Create a dictionary using unordered_map to store country-capital pairs (Add, Search, Display).

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

int main() {
    unordered_map<string, string> dict;
    int ch; string c, cap;
    do {
        cout << "\n1.Add  2.Search  3.Display All  4.Exit\nChoice: "; cin >> ch;
        if (ch == 1) {
            cout << "Enter Country and Capital: "; cin >> c >> cap;
            dict[c] = cap;
        } else if (ch == 2) {
            cout << "Enter Country: "; cin >> c;
            if (dict.count(c)) cout << "Capital: " << dict[c] << endl;
            else cout << "Not Found!\n";
        } else if (ch == 3) {
            for (auto &p : dict) cout << p.first << " -> " << p.second << endl;
        }
    } while (ch != 4);
    return 0;
}
```

Slip 20

Q.1) Read two integers and divide the first by the second using exception handling for division by zero.

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two integers: "; cin >> a >> b;
    try {
        if (b == 0) throw 0;
        cout << "Division: " << (float)a / b << endl;
    } catch (int) {
        cout << "Exception: Division by zero!\n";
    }
    return 0;
}
```

Q.2) Multilevel inheritance: Train $\rightarrow$ Route $\rightarrow$ Reservation.
Q.2 Ans is identical to Slip 17 Q.2 – simply replace Flight with Train (Train_no, Train_Name), add route_id in Route, and prompt the user for the specified Train_Class to filter in main().

Slip 21
Q.1) Class Integer that overloads binary operators +, -, *, /.

```
#include <iostream>
using namespace std;

class Integer {
    int n;
public:
    Integer(int v = 0) : n(v) {}
    Integer operator+(Integer b) { return Integer(n + b.n); }
    Integer operator-(Integer b) { return Integer(n - b.n); }
    Integer operator*(Integer b) { return Integer(n * b.n); }
    Integer operator/(Integer b) { return Integer(n / b.n); }
    void show() { cout << n << endl; }
};

int main() {
    Integer a(20), b(5);
    (a + b).show(); (a - b).show(); (a * b).show(); (a / b).show();
    return 0;
}
```

Q.2) Base class Media (title, price), derived classes Book and Tape. Use virtual functions to display details.

```
#include <iostream>
#include <string>
using namespace std;

class Media {
protected:
    string title; float price;
public:
    Media(string t, float p) : title(t), price(p) {}
    virtual void display() = 0;
};

class Book : public Media {
    int pages;
public:
    Book(string t, float p, int pg) : Media(t, p), pages(pg) {}
    void display() override {
        cout << "Book: " << title << " | Rs." << price << " | Pages: " << pages <<
endl;
    }
};
```

```
class Tape : public Media {
    float time;
public:
    Tape(string t, float p, float tm) : Media(t, p), time(tm) {}
    void display() override {
        cout << "Tape: " << title << " | Rs." << price << " | Time: " << time << "
mins\n";
    }
};

int main() {
    Book b("CPP_Primer", 550, 400);
    Tape t("Audio_Course", 300, 60.5);
    Media *m1 = &b, *m2 = &t;
    m1->display(); m2->display();
    return 0;
}
```

Slip 22

Q.1) Read a text file and count Uppercase, Lowercase, Digits, and Spaces.

```
#include <iostream>
#include <fstream>
#include <cctype>
using namespace std;

int main() {
    ifstream fin("sample.txt");
    char ch; int u = 0, l = 0, d = 0, s = 0;
    while (fin.get(ch)) {
        if (isupper(ch)) u++;
        else if (islower(ch)) l++;
        else if (isdigit(ch)) d++;
        else if (ch == ' ') s++;
    }
    cout << "Upper: " << u << "\nLower: " << l << "\nDigits: " << d << "\nSpaces: "
<< s << endl;
    fin.close();
    return 0;
}
```

Q.2) Class Point (x, y). Find distance between two points using friend operator-overloading

```
#include <iostream>
#include <cmath>
using namespace std;

class Point {
    float x, y;
public:
```

```
    void accept() { cout << "Enter x and y: "; cin >> x >> y; }
    void display() { cout << "(" << x << ", " << y << ")\n"; }
    friend float operator-(Point p1, Point p2) {
        return sqrt(pow(p2.x - p1.x, 2) + pow(p2.y - p1.y, 2));
    }
};

int main() {
    Point p1, p2;
    p1.accept(); p2.accept();
    cout << "Distance: " << (p1 - p2) << endl;
    return 0;
}
```

Slip 23

Q.1) Count and display the number of lines not starting with alphabet 'C' in a text file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream fin("sample.txt");
    string line; int count = 0;
    while (getline(fin, line)) {
        if (line.empty() || (line[0] != 'C' && line[0] != 'c')) {
            cout << line << endl;
            count++;
        }
    }
    cout << "Lines not starting with 'C': " << count << endl;
    fin.close();
    return 0;
}
```

Q.2) Design class Complex and use a friend function to add two complex numbers.

```
#include <iostream>
using namespace std;

class Complex {
    float r, i;
public:
    Complex(float real = 0, float imag = 0) : r(real), i(imag) {}
    friend Complex add(Complex c1, Complex c2) {
        return Complex(c1.r + c2.r, c1.i + c2.i);
    }
    void display() { cout << r << " + " << i << "i\n"; }
};
```

```
int main() {
    Complex c1(3.5, 2.5), c2(1.5, 4.5);
    Complex c3 = add(c1, c2);
    c3.display();
    return 0;
}
```

Slip 24

Q.1) Class Number with two integer members. Initialize using default and parameterized constructors, and display the maximum of the two numbers.

```
#include <iostream>
using namespace std;

class Number {
    int a, b;
public:
    Number() : a(10), b(20) {}
    Number(int x, int y) : a(x), b(y) {}
    void showMax() { cout << "Max: " << (a > b ? a : b) << endl; }
};

int main() {
    Number n1, n2(45, 30);
    n1.showMax();
    n2.showMax();
    return 0;
}
```

Q.2) Class Employee and a friend function to calculate the average salary from an array of employees

```
#include <iostream>
using namespace std;

class Employee {
    float sal;
public:
    void accept() { cout << "Enter Salary: "; cin >> sal; }
    friend float avgSalary(Employee e[], int n);
};

float avgSalary(Employee e[], int n) {
    float sum = 0;
    for (int i = 0; i < n; i++) sum += e[i].sal;
    return sum / n;
}

int main() {
    int n; cout << "Enter number of employees: "; cin >> n;
    Employee e[n];
    for (int i = 0; i < n; i++) e[i].accept();
```

```
    cout << "Average Salary: " << avgSalary(e, n) << endl;
    return 0;
}
```

Slip 25

Q.1) Class Employee (emp_id, emp_name, basic_salary). Accept data for 5 objects and display employees with salary > 5000.

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
public:
    int id; string name; float sal;
    void accept() { cout << "Enter ID, Name, Salary: "; cin >> id >> name >> sal; }
    void display() { cout << id << " | " << name << " | Rs." << sal << endl; }
};

int main() {
    Employee e[5];
    for (int i = 0; i < 5; i++) e[i].accept();
    cout << "\nEmployees with Salary > 5000:\n";
    for (int i = 0; i < 5; i++)
        if (e[i].sal > 5000) e[i].display();
    return 0;
}
```

Q.2) Class Student (roll_no, name, marks). Use a friend function to display the student with the highest marks among two students.

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int roll; string name; float marks;
public:
    void accept() { cout << "Enter Roll, Name, Marks: "; cin >> roll >> name >> marks; }
    friend void highest(Student s1, Student s2);
};

void highest(Student s1, Student s2) {
    Student top = (s1.marks > s2.marks) ? s1 : s2;
    cout << "Highest Marks: " << top.roll << " | " << top.name << " (" << top.marks << ")\n";
}

int main() {
    Student s1, s2;
    s1.accept(); s2.accept();
```

```
    highest(s1, s2);
    return 0;
}
```